

Build a Recipe API

We need to build the first version of our cookbook platform's backend. The frontend team is waiting on a Recipe API before they can start integration, so we need to get the core endpoints working. The project skeleton is already set up — your job is to implement the business logic inside the existing structure.

What's already provided:

- `Recipe.java` — model with `id`, `title`, `category`, and `rating` fields
- `RecipeRepository.java` — JPA repository interface, ready to use
- `DataLoader.java` — seeds 5 recipes on startup so you have data to work with
- `RecipeController.java` — controller with stubbed methods and TODOs, this is where you work

Specifically, you are expected to:

1. Implement the POST endpoint so recipes can be saved and returned with HTTP 201 Created.
2. Add validation to the POST endpoint so requests with a blank title, blank category, or a rating outside 1–5 are rejected with HTTP 400 Bad Request before any data is saved.
3. Implement the GET endpoint that returns all saved recipes.
4. Extend the GET all endpoint to accept an optional `?category=` filter so callers can request only recipes of a specific type.
5. Implement the GET endpoint that retrieves a single recipe by its ID, returning HTTP 404 Not Found when the ID does not exist.
6. Extend the GET all endpoint to also accept an optional `?rating=` filter, and support both filters being used together.

Deliverables:

- `POST /api/recipes` saves a recipe and returns 201 with the saved entity including its generated ID.

- `POST /api/recipes` rejects a blank title, blank category, or invalid rating with a 400 response.
- `GET /api/recipes` returns all recipes.
- `GET /api/recipes?category=pasta` returns only recipes matching that category.
- `GET /api/recipes/{id}` returns a single recipe or 404 if not found.
- `GET /api/recipes?rating=5` returns only recipes with that star rating.
- `GET /api/recipes?category=pasta&rating=4` supports both filters combined.

Testing Endpoints

Add a recipe:

```
curl -i -X POST http://localhost:8080/api/recipes \
  -H "Content-Type: application/json" \
  -d '{"title": "Mushroom Risotto", "category": "pasta", "rating": 4}' | cat
```

Add with blank title — expect 400:

```
curl -i -X POST http://localhost:8080/api/recipes \
  -H "Content-Type: application/json" \
  -d '{"title": "", "category": "pasta", "rating": 4}' | cat
```

Add with invalid rating — expect 400:

```
curl -i -X POST http://localhost:8080/api/recipes \
  -H "Content-Type: application/json" \
  -d '{"title": "Greek Salad", "category": "salad", "rating": 9}' | cat
```

Get all recipes:

```
curl -s http://localhost:8080/api/recipes | cat
```

Filter by category:

```
curl -s "http://localhost:8080/api/recipes?category=salad" | cat
```

Get by ID:

```
curl -s http://localhost:8080/api/recipes/1 | cat
```

Get by ID — expect 404:

```
curl -s http://localhost:8080/api/recipes/9999 | cat
```

Filter by rating:

```
curl -s "http://localhost:8080/api/recipes?rating=5" | cat
```

Both filters combined:

```
curl -s "http://localhost:8080/api/recipes?category=pasta&rating=4" | cat
```